Homework 5

Zara Zarzour

5/29/2021

a. Data gathering and integration

The first part is to get the data you will use. You may use anything that has not been used in an assignment or tutorial. It must have at least 100 data points and must include both numerical and categorial (or ordinal) variables. I recommend keeping this relatively straightforward because data cleaning can take a lot of time if you choose a large, messy dataset. Kaggle (https://www.kaggle.com/datasets) and the University of California at Irvine (UCI) (https://archive.ics.uci.edu/ml/index.php) maintain collections of datasets, some even telling you if they are good examples for testing specific machine learning techniques. You may also choose to join together more than one dataset, for example to merge data on health outcomes by US state with a dataset on food statistics per state. Merging data is not required and will earn you a bonus point in this step.

```
#import insurance dataset
library(readxl)
insurance <- read_csv("C:/Users/[redacted]/OneDrive/Desktop/DePaul/DSC 441/Hwk 5/insurance.csv")
```

```
##
## -- Column specification --------------------------------------------------------
## cols(
##   age = col_double(),
##   sex = col_character(),
##   bmi = col_double(),
##   children = col_double(),
##   smoker = col_character(),
##   region = col_character(),
##   charges = col_double()
## )
```

```
head(insurance)
```

```
## # A tibble: 6 x 7
##     age sex      bmi children smoker region    charges
##   <dbl> <chr>  <dbl>    <dbl> <chr>  <chr>       <dbl>
## 1    19 female  27.9        0 yes    southwest  16885.
## 2    18 male    33.8        1 no     southeast   1726.
## 3    28 male    33          3 no     southeast   4449.
## 4    33 male    22.7        0 no     northwest  21984.
## 5    32 male    28.9        0 no     northwest   3867.
## 6    31 female  25.7        0 no     southeast   3757.
```

I've decided to use the insurance dataset to predict the smokers based on the data provided I'd also like to work off of the hypothesis that charges will likely increase if you're a smoker. This will set the tone for the rest of my analysis.

b. Data Exploration

Using data exploration to understand what is happening is important throughout the pipeline, and is not limited to this step. However, it is important to use some exploration early on to make sure you understand your data. You must at least consider the distributions of each variable and at least some of the relationships between pairs of variables.

```
summary(insurance)
```

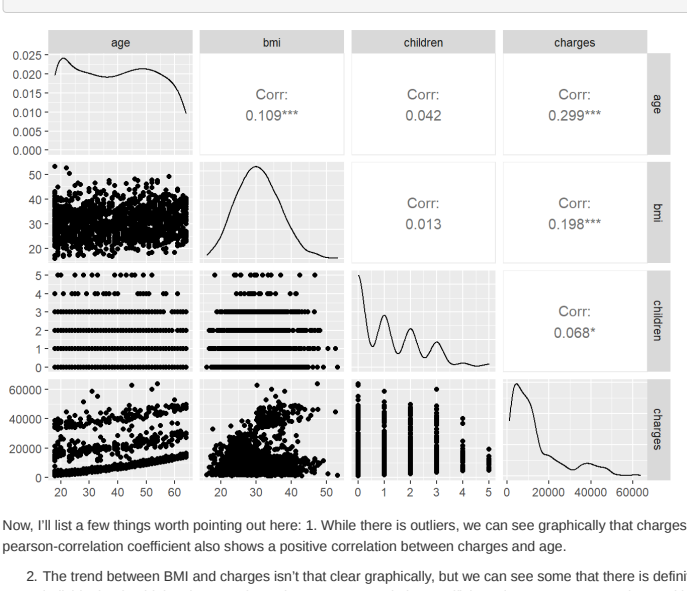
```
##       age            sex                 bmi           children    
##  Min.   :18.00   Length:1338        Min.   :15.96   Min.   :0.000  
##  1st Qu.:27.00   Class :character   1st Qu.:26.30   1st Qu.:0.000  
##  Median :39.00   Mode  :character   Median :30.40   Median :1.000  
##  Mean   :39.21                      Mean   :30.66   Mean   :1.095  
##  3rd Qu.:51.00                      3rd Qu.:34.69   3rd Qu.:2.000  
##  Max.   :64.00                      Max.   :53.13   Max.   :5.000  
##     smoker             region             charges     
##  Length:1338        Length:1338        Min.   : 1122  
##  Class :character   Class :character   1st Qu.: 4740  
##  Mode  :character   Mode  :character   Median : 9382  
##                                        Mean   :13270  
##                                        3rd Qu.:16640  
##                                        Max.   :63770
```

There are no NA's in this dataset. Sex, region, and smokers are categorical, while the rest of the dataset is numeric.

Now, let's perform some more exploratory tasks and find existing relationships.

First, I'll create a scatterplot matrix of the numerical variables.

```
insurance %>% select(age, bmi, children, charges) %>% ggpairs()
```



Now, I'll list a few things worth pointing out here: 1. While there is outliers, we can see graphically that charges generally increase with age. The pearson-correlation coefficient also shows a positive correlation between charges and age.

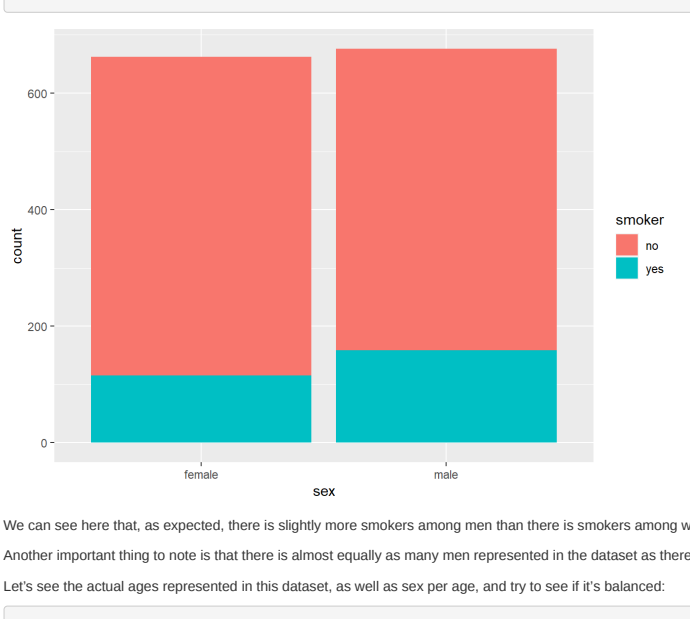
2. The trend between BMI and charges isn't that clear graphically, but we can see some that there is definitely an increase in charges for some individuals after hitting the age of 30. The pearson-correlation coefficient also proves a somewhat positive correlation. This will be interesting to discover later based on sex and smoker-status.

3. Note the somewhat positive correlation number between age and BMI, although this is unclear graphically.

Now, it will be interesting to view the number of female to men whom are smokers to nonsmokers:

```
# Convert to dataframe
df <- as.data.frame(insurance)

# Create ggplot object
p <- ggplot(insurance, aes(x=sex, fill=smoker))
p + geom_bar(position="stack")
```

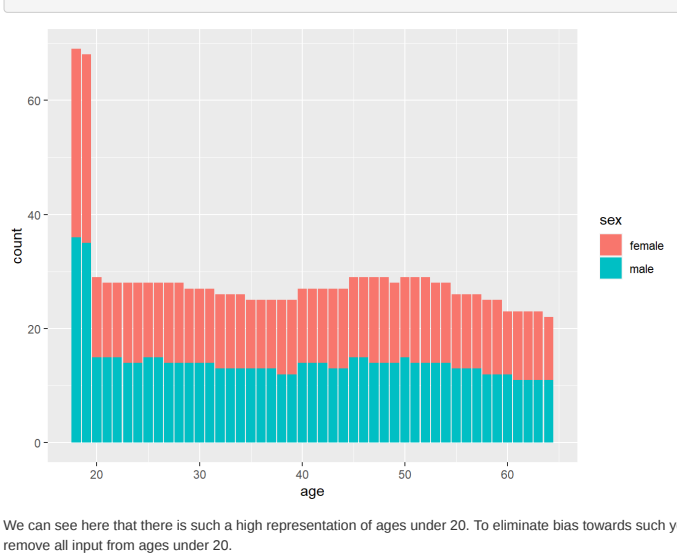


We can see here that, as expected, there is slightly more smokers among men than there is smokers among women.

Another important thing to note is that there is almost equally as many men represented in the dataset as there is women.

Let's see the actual ages represented in this dataset, as well as sex per age, and try to see if it's balanced:

```
p <- ggplot(df, aes(x=age, fill=sex))
p + geom_bar(position="stack")
```



We can see here that there is such a high representation of ages under 20. To eliminate bias towards such younger age groups, it may help to remove all input from ages under 20.

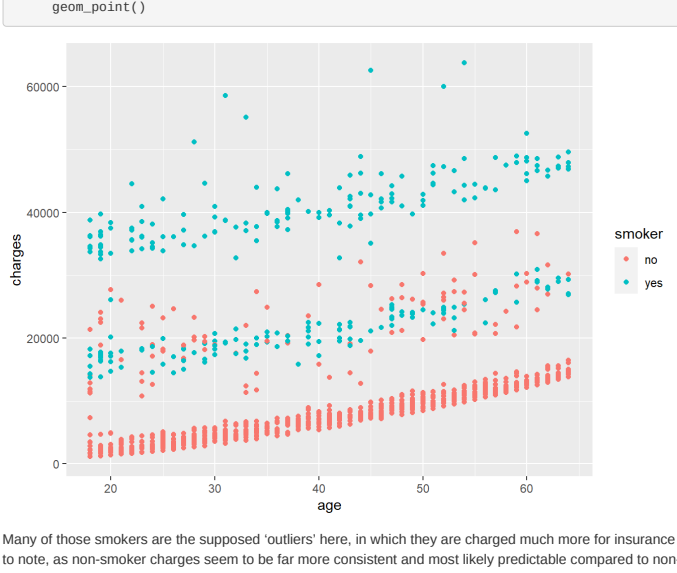
Let's quickly view the number of smokers here:

```
insurance %>% group_by(smoker) %>% summarise("count"=n())
```

```
## # A tibble: 2 x 2
##   smoker count
##   <chr>  <int>
## 1 no      1064
## 2 yes      274
```

We have 274 smokers in this dataset. Now let's look at this graphically:

```
ggplot(insurance, aes(x=age, y=charges, color=smoker)) +
    geom_point()
```



Many of those smokers are the supposed 'outliers' here, in which they are charged much more for insurance than non-smokers. This is important to note, as non-smoker charges seem to be far more consistent and most likely predictable compared to non-smokers.

c.Data Cleaning

Don't forget – this can take a lot of the time of the whole process. Your cleaning process must ensure that there are no missing values and all outliers must be considered. It may be reasonable to just remove rows with missing values, however, if your data or small or that would change the distributions of the variables, that will not be adequate and you will need to consider other options, as discussed in the modules on cleaning. Depending on your data and what you plan to do with it, you may also need to apply other processes we discussed. For example, clean up strings for consistency, deal with date formatting, change variable types between categorical and numeric, bin, smooth, group, aggregate or reshape. Make the case with visualization or by showing resulting summary statistics that your data are clean enough to continue with your analysis.

Now let's do some data cleaning. Fortunately, this dataset seems to be mostly straightforward with no NA's or missing data as we saw in the previous step.

```
summary(insurance)
```

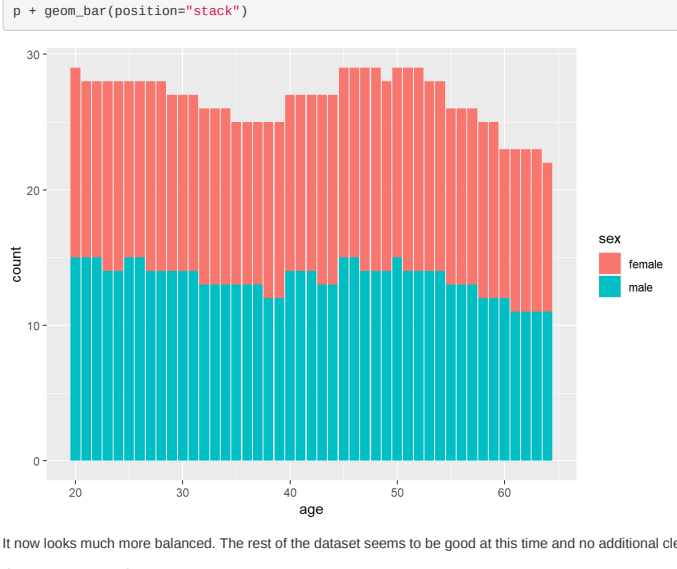
```
##       age            sex                 bmi           children    
##  Min.   :18.00   Length:1338        Min.   :15.96   Min.   :0.000  
##  1st Qu.:27.00   Class :character   1st Qu.:26.30   1st Qu.:0.000  
##  Median :39.00   Mode  :character   Median :30.40   Median :1.000  
##  Mean   :39.21                      Mean   :30.66   Mean   :1.095  
##  3rd Qu.:51.00                      3rd Qu.:34.69   3rd Qu.:2.000  
##  Max.   :64.00                      Max.   :53.13   Max.   :5.000  
##     smoker             region             charges     
##  Length:1338        Length:1338        Min.   : 1122  
##  Class :character   Class :character   1st Qu.: 4740  
##  Mode  :character   Mode  :character   Median : 9382  
##                                        Mean   :13270  
##                                        3rd Qu.:16640  
##                                        Max.   :63770
```

Although, we did notice that there may be a bias towards ages younger than 20, so I will not remove all datapoints of those younger than 20.

```
#exclude rows using subset function with condition of including ages greater than or equal to 20
insurance <- subset(insurance, age >= 20)
```

Now, let's see how this looks graphically:

```
p <- ggplot(insurance, aes(x=age, fill=sex))
p + geom_bar(position="stack")
```



It now looks much more balanced. The rest of the dataset seems to be good at this time and no additional cleaning is necessary.

d.Data Preprocessing

In some cases, preprocessing is absolutely necessary. It is rarely a bad idea. Make the case for what is and is not necessary given what you plan to do with the data. This could include making dummy variables, applying normalization, binning and/or smoothing, and other transformations (see course module).

I think we'll most likely use dummy variables, so I will go ahead and create those now. But first, I'll remove region as it may not be necessary.

```
#remove region's column
df <- insurance
predictors <- df %>% select(-c(region))
head(predictors)
```

```
## # A tibble: 6 x 6
##     age sex      bmi children smoker charges
##   <dbl> <chr>  <dbl>    <dbl> <chr>    <dbl>
## 1    28 male    33          3 no       4449.
## 2    33 male    22.7        0 no      21984.
## 3    32 male    28.9        0 no       3867.
## 4    31 female  25.7        0 no       3757.
## 5    46 female  33.4        1 no       8241.
## 6    37 female  27.7        3 no       7282.
```

Now the dummies:

```
#create dummies
dummy <- dummyVars(smoker ~ ., data = predictors)
dummies <- as.data.frame(predict(dummy, newdata = predictors))
head(dummies)
```

```
##   age sexfemale sexmale    bmi children   charges
## 1  28         0       1 33.000        3  4449.462
## 2  33         0       1 22.705        0 21984.471
## 3  32         0       1 28.880        0  3866.855
## 4  31         1       0 25.740        0  3756.622
## 5  46         1       0 33.440        1  8240.590
## 6  37         1       0 27.740        3  7281.506
```

e. Clustering Remove any labels from your data and use clustering to discover any built-in structure. Use an appropriate method to determine the number of clusters. If your data have labels, compare the clusters to those labels. If not, visualize the clustering results by making a PCA projection and coloring the points by cluster assignment. Note that PCA only works for numerical variables, so if your data have just a few categorical, you may skip them. If there are many, use dummy variables or choose a different method for making a projection. One way is to make the distance matrix first (we covered a method for distance matrices using categorical variables in the clustering tutorial) and then apply PCA to that matrix. This is actually a way to calculate an MDS projection, a very popular method.

I'll rename the dummies as predictors:

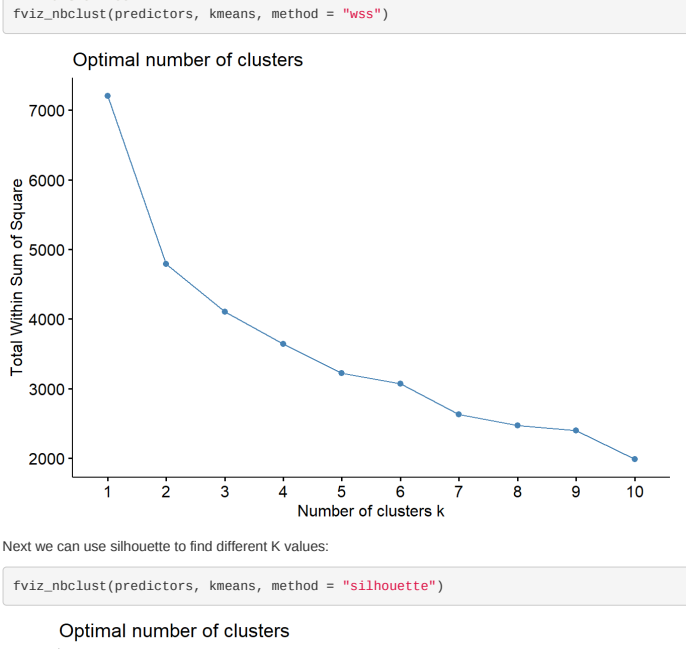
```
predictors <- dummies
```

Now, let's use K-means to cluster the data:

```
# Set seed
set.seed(123)
# Center scale allows us to standardize the data
preproc <- preProcess(predictors, method=c("center", "scale"))
# We have to call predict to fit our data based on preprocessing
predictors <- predict(preproc, predictors)
```

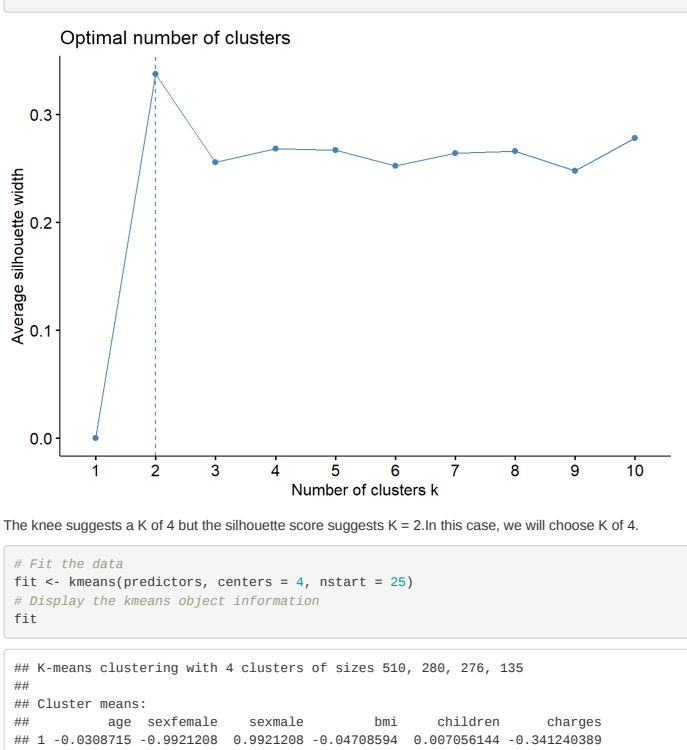
Now find the knee:

```
# Find the knee
fviz_nbclust(predictors, kmeans, method = "wss")
```



Next we can use silhouette to find different K values:

```
fviz_nbclust(predictors, kmeans, method = "silhouette")
```



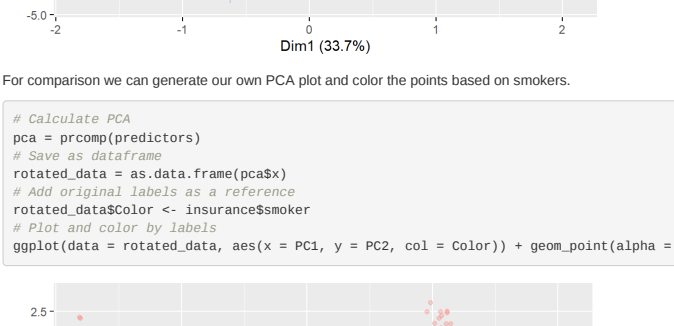
The knee suggests a K of 4 but the silhouette score suggests K = 2.In this case, we will choose K of 4.

```
# Fit the data
fit <- kmeans(predictors, centers = 4, nstart = 25)
# Display the kmeans object information
fit
```

```
## K-means clustering with 4 clusters of sizes 510, 280, 276, 135
##
## Cluster means:
##          age  sexfemale    sexmale         bmi     children      charges
## 1 -0.0308715 -0.9921208  0.9921208 -0.04708594  0.007056144 -0.341240389
## 2  0.8566914  1.0071025 -1.0071025  0.06847007 -0.072196373 -0.003991792
## 3 -0.8611210  1.0071025 -1.0071025 -0.35858721  0.027194427 -0.498706273
## 4  0.1002982 -0.3997584  0.3997584  0.76897985  0.067486588  2.316986898
##
## Clustering vector:
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16 
##    1    1    1    3    2    3    1    2    1    2    1    2    4    2    1    1 
##   17   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32 
##    4    2    3    4    1    2    2    2    1    4    4    1    4    2    1    4 
##   33   34   35   36   37   38   39   40   41   42   43   44   45   46   47   48 
##    4    3    3    1    3    1    1    3    2    4    3    1    4    3    4    2 
##   49   50   51   52   53   54   55   56   57   58   59   60   61   62   63   64 
##    2    3    1    1    1    3    3    2    1    2    1    3    1    1    1    1 
##   65   66   67   68   69   70   71   72   73   74   75   76   77   78   79   80 
##    1    3    1    3    2    1    2    4    2    4    1    4    2    2    2    3 
##   81   82   83   84   85   86   87   88   89   90   91   92   93   94   95   96 
##    2    4    1    4    3    2    1    1    1    3    1    2    3    1    1    1 
##   97   98   99  100  101  102  103  104  105  106  107  108  109  110  111  112 
##    4    1    2    1    3    1    4    1    3    2    3    1    3    4    2    3 
##  113  114  115  116  117  118  119  120  121  122  123  124  125  126  127  128 
##    2    3    1    2    2    2    3    3    1    2    3    1    1    1    1    1 
##  129  130  131  132  133  134  135  136  137  138  139  140  141  142  143  144 
##    3    4    2    2    1    1    3    2    3    1    1    4    2    2    1    3 
##  145  146  147  148  149  150  151  152  153  154  155  156  157  158  159  160 
##    1    1    3    3    1    1    1    3    3    4    4    1    1    2    3    1 
##  161  162  163  164  165  166  167  168  169  170  171  172  173  174  175  176 
##    2    1    4    3    3    3    3    1    3    1    2    3    2    2    2    2 
##  177  178  179  180  181  182  183  184  185  186  187  188  189  190  191  192 
##    2    4    1    3    1    1    2    1    1    1    3    2    2    2    1    1 
##  193  194  195  196  197  198  199  200  201  202  203  204  205  206  207  208 
##    3    3    3    2    1    1    1    1    2    3    1    2    1    1    1    3 
##  209  210  211  212  213  214  215  216  217  218  219  220  221  222  223  224 
## [illegible]
##  225  226  227  228  229  230  231  232  233  234  235  236  237  238  239  240 
## [illegible]
##  241  242  243  244  245  246  247  248  249  250  251  252  253  254  255  256 
## [illegible]
##  257  258  259  260  261  262  263  264  265  266  267  268  269  270  271  272 
## [illegible]
##  273  274  275  276  277  278  279  280  281  282  283  284  285  286  287  288 
## [illegible]
##  289  290  291  292  293  294  295  296  297  298  299  300  301  302  303  304 
## [illegible]
##  305  306  307  308  309  310  311  312  313  314  315  316  317  318  319  320 
## [illegible]
##  321  322  323  324  325  326  327  328  329  330  331  332  333  334  335  336 
## [illegible]
##  337  338  339  340  341  342  343  344  345  346  347  348  349  350  351  352 
## [illegible]
##  353  354  355  356  357  358  359  360  361  362  363  364  365  366  367  368 
## [illegible]
##  369  370  371  372  373  374  375  376  377  378  379  380  381  382  383  384 
## [illegible]
##  385  386  387  388  389  390  391  392  393  394  395  396  397  398  399  400 
## [illegible]
##  401  402  403  404  405  406  407  408  409  410  411  412  413  414  415  416 
## [illegible]
##  417  418  419  420  421  422  423  424  425  426  427  428  429  430  431  432 
## [illegible]
##  433  434  435  436  437  438  439  440  441  442  443  444  445  446  447  448 
## [illegible]
##  449  450  451  452  453  454  455  456  457  458  459  460  461  462  463  464 
## [illegible]
##  465  466  467  468  469  470  471  472  473  474  475  476  477  478  479  480 
## [illegible]
##  481  482  483  484  485  486  487  488  489  490  491  492  493  494  495  496 
## [illegible]
##  497  498  499  500  501  502  503  504  505  506  507  508  509  510  511  512 
## [illegible]
##  513  514  515  516  517  518  519  520  521  522  523  524  525  526  527  528 
## [illegible]
##  529  530  531  532  533  534  535  536  537  538  539  540  541  542  543  544 
## [illegible]
##  545  546  547  548  549  550  551  552  553  554  555  556  557  558  559  560 
## [illegible]
##  561  562  563  564  565  566  567  568  569  570  571  572  573  574  575  576 
## [illegible]
##  577  578  579  580  581  582  583  584  585  586  587  588  589  590  591  592 
## [illegible]
##  593  594  595  596  597  598  599  600  601  602  603  604  605  606  607  608 
## [illegible]
##  609  610  611  612  613  614  615  616  617  618  619  620  621  622  623  624 
## [illegible]
##  625  626  627  628  629  630  631  632  633  634  635  636  637  638  639  640 
## [illegible]
##  641  642  643  644  645  646  647  648  649  650  651  652  653  654  655  656 
## [illegible]
##  657  658  659  660  661  662  663  664  665  666  667  668  669  670  671  672 
## [illegible]
##  673  674  675  676  677  678  679  680  681  682  683  684  685  686  687  688 
## [illegible]
##  689  690  691  692  693  694  695  696  697  698  699  700  701  702  703  704 
## [illegible]
##  705  706  707  708  709  710  711  712  713  714  715  716  717  718  719  720 
## [illegible]
##  721  722  723  724  725  726  727  728  729  730  731  732  733  734  735  736 
## [illegible]
##  737  738  739  740  741  742  743  744  745  746  747  748  749  750  751  752 
## [illegible]
##  753  754  755  756  757  758  759  760  761  762  763  764  765  766  767  768 
## [illegible]
##  769  770  771  772  773  774  775  776  777  778  779  780  781  782  783  784 
## [illegible]
##  785  786  787  788  789  790  791  792  793  794  795  796  797  798  799  800 
## [illegible]
##  801  802  803  804  805  806  807  808  809  810  811  812  813  814  815  816 
## [illegible]
##  817  818  819  820  821  822  823  824  825  826  827  828  829  830  831  832 
## [illegible]
##  833  834  835  836  837  838  839  840  841  842  843  844  845  846  847  848 
## [illegible]
##  849  850  851  852  853  854  855  856  857  858  859  860  861  862  863  864 
## [illegible]
##  865  866  867  868  869  870  871  872  873  874  875  876  877  878  879  880 
## [illegible]
##  881  882  883  884  885  886  887  888  889  890  891  892  893  894  895  896 
## [illegible]
##  897  898  899  900  901  902  903  904  905  906  907  908  909  910  911  912 
## [illegible]
##  913  914  915  916  917  918  919  920  921  922  923  924  925  926  927  928 
## [illegible]
##  929  930  931  932  933  934  935  936  937  938  939  940  941  942  943  944 
## [illegible]
##  945  946  947  948  949  950  951  952  953  954  955  956  957  958  959  960 
## [illegible]
##  961  962  963  964  965  966  967  968  969  970  971  972  973  974  975  976 
## [illegible]
##  977  978  979  980  981  982  983  984  985  986  987  988  989  990  991  992 
## [illegible]
##  993  994  995  996  997  998  999 1000 1001 1002 1003 1004 1005 1006 1007 1008 
## [illegible]
## 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 
## [illegible]
## 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 
## [illegible]
## 1041 1042 1043 1044 1045 1046 1047 1048 1049 1050 1051 1052 1053 1054 1055 1056 
## [illegible]
## 1057 1058 1059 1060 1061 1062 1063 1064 1065 1066 1067 1068 1069 1070 1071 1072 
## [illegible]
## 1073 1074 1075 1076 1077 1078 1079 1080 1081 1082 1083 1084 1085 1086 1087 1088 
## [illegible]
## 1089 1090 1091 1092 1093 1094 1095 1096 1097 1098 1099 1100 1101 1102 1103 1104 
## [illegible]
## 1105 1106 1107 1108 1109 1110 1111 1112 1113 1114 1115 1116 1117 1118 1119 1120 
## [illegible]
## 1121 1122 1123 1124 1125 1126 1127 1128 1129 1130 1131 1132 1133 1134 1135 1136 
## [illegible]
## 1137 1138 1139 1140 1141 1142 1143 1144 1145 1146 1147 1148 1149 1150 1151 1152 
## [illegible]
## 1153 1154 1155 1156 1157 1158 1159 1160 1161 1162 1163 1164 1165 1166 1167 1168 
## [illegible]
## 1169 1170 1171 1172 1173 1174 1175 1176 1177 1178 1179 1180 1181 1182 1183 1184 
## [illegible]
## 1185 1186 1187 1188 1189 1190 1191 1192 1193 1194 1195 1196 1197 1198 1199 1200 
## [illegible]
## 1201 
##    2 
##
## Within cluster sum of squares by cluster:
## [1] 1692.4575  682.5247  684.7234  580.4621
##  (between_SS / total_SS =  49.4 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

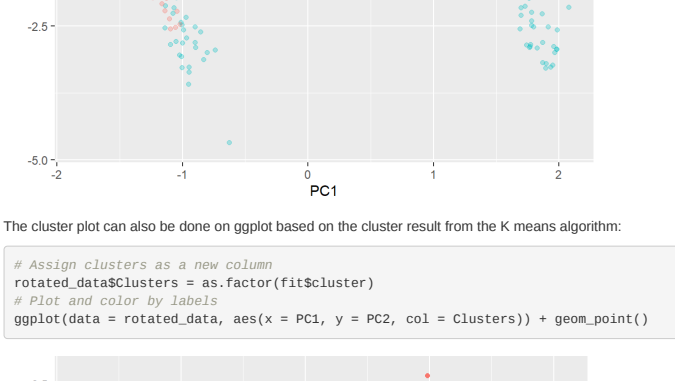
Based on this, we can visualize how the clusters were formed:

```
# Display the cluster plot
fviz_cluster(fit, data = predictors)
```



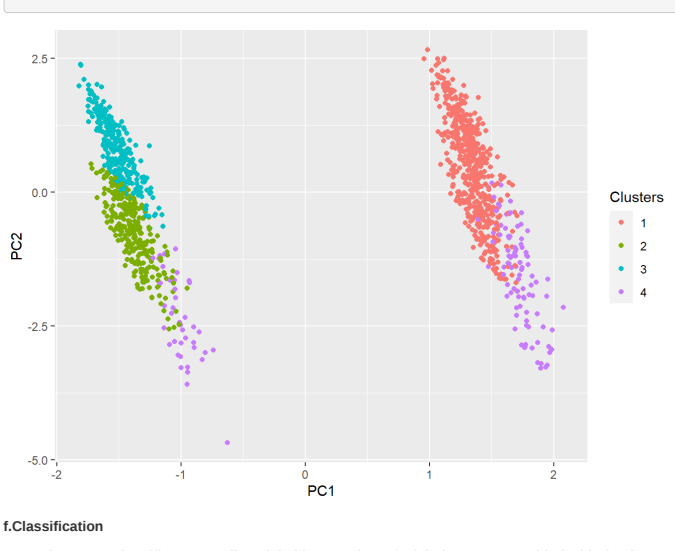
For comparison we can generate our own PCA plot and color the points based on smokers.

```
# Calculate PCA
pca = prcomp(predictors)
# Save as dataframe
rotated_data = as.data.frame(pca$x)
# Add original labels as a reference
rotated_data$Color <- insurance$smoker
# Plot and color by labels
ggplot(data = rotated_data, aes(x = PC1, y = PC2, col = Color)) + geom_point(alpha = 0.3)
```



The cluster plot can also be done on ggplot based on the cluster result from the K means algorithm:

```
# Assign clusters as a new column
rotated_data$Clusters = as.factor(fit$cluster)
# Plot and color by labels
ggplot(data = rotated_data, aes(x = PC1, y = PC2, col = Clusters)) + geom_point()
```



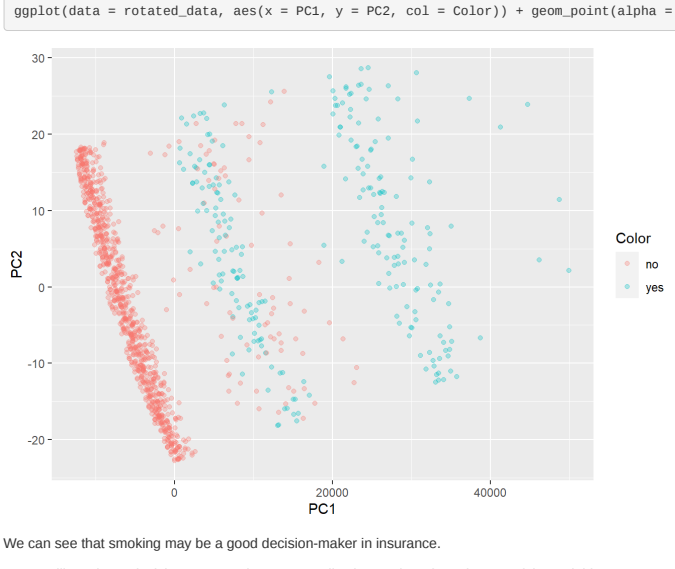
f.Classification

Use at least two classifiers to predict a label in your data. If a label was not provided with the data, use the clustering from the previous part. Follow the process for choosing the best parameters for your choice of classifier. Compare the accuracy of the two.

First, I used PCA and dummy variables to create a projection of the data to 2D and to show a scatterplot with color showing the 'yes' or 'no' for smoking.

```
set.seed(123)
#calculate PCA
pca = prcomp(dummies)
#save as data frame
rotated_data = as.data.frame(pca$x)

#add original label 'smoker' as a reference
rotated_data$Color <- insurance$smoker
#plot and color the labels based on 'yes' or 'no' for smoking
ggplot(data = rotated_data, aes(x = PC1, y = PC2, col = Color)) + geom_point(alpha = 0.3)
```



We can see that smoking may be a good decision-maker in insurance.

Now, I will use kNN, decision trees, and SVM to predict the smokers from the rest of the variables.

KNN:

```
set.seed(123)
ctrl <- trainControl(method="cv", number = 10)
knnFit <- train(smoker ~ ., data = insurance,
                method = "knn",
                trControl = ctrl,
                preProcess = c("center","scale"),
                tuneLength = 15)

knnFit
```

```
## k-Nearest Neighbors
##
## 1201 samples
##    6 predictor
##    2 classes: 'no', 'yes'
##
## Pre-processing: centered (8), scaled (8)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1080, 1081, 1080, 1081, 1081, 1082, ...
## Resampling results across tuning parameters:
##
##   k   Accuracy   Kappa
##    5  0.9309628  0.7802508
##    7  0.9201368  0.7340324
##    9  0.9126563  0.6999024
##   11  0.9084618  0.6733136
##   13  0.9034824  0.6473479
##   15  0.9051353  0.6504538
##   17  0.9043298  0.6459957
##   19  0.9043299  0.6426396
##   21  0.9051702  0.6450476
##   23  0.9060105  0.6475427
##   25  0.9051772  0.6438390
##   27  0.9043369  0.6402444
##   29  0.9043369  0.6402444
##   31  0.9043438  0.6393377
##   33  0.9035104  0.6356107
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 5.
```

For K=5:

```
fit <- kmeans(dummies, centers = 5, nstart = 25)
# Display the kmeans object information
fit
```

```
## K-means clustering with 5 clusters of sizes 57, 420, 477, 158, 89
##
## Cluster means:
##        age sexfemale   sexmale      bmi children   charges
## 1 51.28070 0.4210526 0.5789474 37.26009 1.245614 47388.517
## 2 52.12143 0.5404762 0.4595238 30.96919 1.154762 11317.146
## 3 31.83019 0.5010482 0.4989518 30.19259 1.140461  4776.254
## 4 41.82278 0.4873418 0.5126582 28.01187 1.208861 22379.800
## 5 37.30337 0.3258427 0.6741573 33.29107 1.280899 37476.293
##
## Clustering vector:
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16 
##    3    4    3    3    2    3    3    4    3    4    3    2    5    3    3    2 
##   17   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32 
##    5    2    3    5    3    2    2    2    3    5    5    2    1    2    3    5 
##   33   34   35   36   37   38   39   40   41   42   43   44   45   46   47   48 
##    1    3    3    3    3    3    4    3    2    5    3    4    5    2    1    2 
##   49   50   51   52   53   54   55   56   57   58   59   60   61   62   63   64 
## [illegible]
##   65   66   67   68   69   70   71   72   73   74   75   76   77   78   79   80 
## [illegible]
##   81   82   83   84   85   86   87   88   89   90   91   92   93   94   95   96 
## [illegible]
##   97   98   99  100  101  102  103  104  105  106  107  108  109  110  111  112 
## [illegible]
##  113  114  115  116  117  118  119  120  121  122  123  124  125  126  127  128 
## [illegible]
##  129  130  131  132  133  134  135  136  137  138  139  140  141  142  143  144 
## [illegible]
##  145  146  147  148  149  150  151  152  153  154  155  156  157  158  159  160 
## [illegible]
##  161  162  163  164  165  166  167  168  169  170  171  172  173  174  175  176 
## [illegible]
##  177  178  179  180  181  182  183  184  185  186  187  188  189  190  191  192 
## [illegible]
##  193  194  195  196  197  198  199  200  201  202  203  204  205  206  207  208 
## [illegible]
##  209  210  211  212  213  214  215  216  217  218  219  220  221  222  223  224 
## [illegible]
##  225  226  227  228  229  230  231  232  233  234  235  236  237  238  239  240 
## [illegible]
##  241  242  243  244  245  246  247  248  249  250  251  252  253  254  255  256 
## [illegible]
##  257  258  259  260  261  262  263  264  265  266  267  268  269  270  271  272 
## [illegible]
##  273  274  275  276  277  278  279  280  281  282  283  284  285  286  287  288 
## [illegible]
##  289  290  291  292  293  294  295  296  297  298  299  300  301  302  303  304 
## [illegible]
##  305  306  307  308  309  310  311  312  313  314  315  316  317  318  319  320 
## [illegible]
##  321  322  323  324  325  326  327  328  329  330  331  332  333  334  335  336 
## [illegible]
##  337  338  339  340  341  342  343  344  345  346  347  348  349  350  351  352 
## [illegible]
##  353  354  355  356  357  358  359  360  361  362  363  364  365  366  367  368 
## [illegible]
##  369  370  371  372  373  374  375  376  377  378  379  380  381  382  383  384 
## [illegible]
##  385  386  387  388  389  390  391  392  393  394  395  396  397  398  399  400 
## [illegible]
##  401  402  403  404  405  406  407  408  409  410  411  412  413  414  415  416 
## [illegible]
##  417  418  419  420  421  422  423  424  425  426  427  428  429  430  431  432 
## [illegible]
##  433  434  435  436  437  438  439  440  441  442  443  444  445  446  447  448 
## [illegible]
##  449  450  451  452  453  454  455  456  457  458  459  460  461  462  463  464 
## [illegible]
##  465  466  467  468  469  470  471  472  473  474  475  476  477  478  479  480 
## [illegible]
##  481  482  483  484  485  486  487  488  489  490  491  492  493  494  495  496 
## [illegible]
##  497  498  499  500  501  502  503  504  505  506  507  508  509  510  511  512 
## [illegible]
##  513  514  515  516  517  518  519  520  521  522  523  524  525  526  527  528 
## [illegible]
##  529  530  531  532  533  534  535  536  537  538  539  540  541  542  543  544 
## [illegible]
##  545  546  547  548  549  550  551  552  553  554  555  556  557  558  559  560 
## [illegible]
##  561  562  563  564  565  566  567  568  569  570  571  572  573  574  575  576 
## [illegible]
##  577  578  579  580  581  582  583  584  585  586  587  588  589  590  591  592 
## [illegible]
##  593  594  595  596  597  598  599  600  601  602  603  604  605  606  607  608 
## [illegible]
##  609  610  611  612  613  614  615  616  617  618  619  620  621  622  623  624 
## [illegible]
##  625  626  627  628  629  630  631  632  633  634  635  636  637  638  639  640 
## [illegible]
##  641  642  643  644  645  646  647  648  649  650  651  652  653  654  655  656 
## [illegible]
##  657  658  659  660  661  662  663  664  665  666  667  668  669  670  671  672 
## [illegible]
##  673  674  675  676  677  678  679  680  681  682  683  684  685  686  687  688 
## [illegible]
##  689  690  691  692  693  694  695  696  697  698  699  700  701  702  703  704 
## [illegible]
##  705  706  707  708  709  710  711  712  713  714  715  716  717  718  719  720 
## [illegible]
##  721  722  723  724  725  726  727  728  729  730  731  732  733  734  735  736 
## [illegible]
##  737  738  739  740  741  742  743  744  745  746  747  748  749  750  751  752 
## [illegible]
##  753  754  755  756  757  758  759  760  761  762  763  764  765  766  767  768 
## [illegible]
##  769  770  771  772  773  774  775  776  777  778  779  780  781  782  783  784 
## [illegible]
##  785  786  787  788  789  790  791  792  793  794  795  796  797  798  799  800 
## [illegible]
##  801  802  803  804  805  806  807  808  809  810  811  812  813  814  815  816 
## [illegible]
##  817  818  819  820  821  822  823  824  825  826  827  828  829  830  831  832 
## [illegible]
##  833  834  835  836  837  838  839  840  841  842  843  844  845  846  847  848 
## [illegible]
##  849  850  851  852  853  854  855  856  857  858  859  860  861  862  863  864 
## [illegible]
##  865  866  867  868  869  870  871  872  873  874  875  876  877  878  879  880 
## [illegible]
##  881  882  883  884  885  886  887  888  889  890  891  892  893  894  895  896 
## [illegible]
##  897  898  899  900  901  902  903  904  905  906  907  908  909  910  911  912 
## [illegible]
##  913  914  915  916  917  918  919  920  921  922  923  924  925  926  927  928 
## [illegible]
##  929  930  931  932  933  934  935  936  937  938  939  940  941  942  943  944 
## [illegible]
##  945  946  947  948  949  950  951  952  953  954  955  956  957  958  959  960 
## [illegible]
##  961  962  963  964  965  966  967  968  969  970  971  972  973  974  975  976 
## [illegible]
##  977  978  979  980  981  982  983  984  985  986  987  988  989  990  991  992 
## [illegible]
##  993  994  995  996  997  998  999 1000 1001 1002 1003 1004 1005 1006 1007 1008 
## [illegible]
## 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 
## [illegible]
## 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 
## [illegible]
## 1041 1042 1043 1044 1045 1046 1047 1048 1049 1050 1051 1052 1053 1054 1055 1056 
## [illegible]
## 1057 1058 1059 1060 1061 1062 1063 1064 1065 1066 1067 1068 1069 1070 1071 1072 
## [illegible]
## 1073 1074 1075 1076 1077 1078 1079 1080 1081 1082 1083 1084 1085 1086 1087 1088 
## [illegible]
## 1089 1090 1091 1092 1093 1094 1095 1096 1097 1098 1099 1100 1101 1102 1103 1104 
## [illegible]
## 1105 1106 1107 1108 1109 1110 1111 1112 1113 1114 1115 1116 1117 1118 1119 1120 
## [illegible]
## 1121 1122 1123 1124 1125 1126 1127 1128 1129 1130 1131 1132 1133 1134 1135 1136 
## [illegible]
## 1137 1138 1139 1140 1141 1142 1143 1144 1145 1146 1147 1148 1149 1150 1151 1152 
## [illegible]
## 1153 1154 1155 1156 1157 1158 1159 1160 1161 1162 1163 1164 1165 1166 1167 1168 
## [illegible]
## 1169 1170 1171 1172 1173 1174 1175 1176 1177 1178 1179 1180 1181 1182 1183 1184 
## [illegible]
## 1185 1186 1187 1188 1189 1190 1191 1192 1193 1194 1195 1196 1197 1198 1199 1200 
## [illegible]
## 1201 
##    4 
##
## Within cluster sum of squares by cluster:
## [1] 1186799633 1908989781 1547477966 1873147993  957818767
##  (between_SS / total_SS =  95.7 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

SVM:

```
#I decided to use grid search here to try different values of C
grid <- expand.grid(C = 10^seq(-5,2,0.5))
train_control = trainControl(method = "cv", number = 10)
# Fit the model
svm_grid <- train(smoker ~., data = insurance, method = "svmLinear",
                  trControl = train_control, tuneGrid = grid)
# View grid search result
svm_grid
```

```
## Support Vector Machines with Linear Kernel
##
## 1201 samples
##    6 predictor
##    2 classes: 'no', 'yes'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1081, 1080, 1081, 1082, 1081, 1080, ...
## Resampling results across tuning parameters:
##
##   C             Accuracy   Kappa
##   1.000000e-05  0.7968471  0.0000000
##   3.162278e-05  0.7968471  0.0000000
##   1.000000e-04  0.7968471  0.0000000
##   3.162278e-04  0.7968471  0.0000000
##   1.000000e-03  0.8109657  0.1036068
##   3.162278e-03  0.8917668  0.6140651
##   1.000000e-02  0.8892737  0.6103669
##   3.162278e-02  0.9175736  0.7274517
##   1.000000e-01  0.9659154  0.9018528
##   3.162278e-01  0.9667557  0.9045667
##   1.000000e+00  0.9659293  0.9024108
##   3.162278e+00  0.9667557  0.9045667
##   1.000000e+01  0.9667557  0.9045667
##   3.162278e+01  0.9667557  0.9045667
##   1.000000e+02  0.9667557  0.9045667
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was C = 0.3162278.
```

Decision Trees:

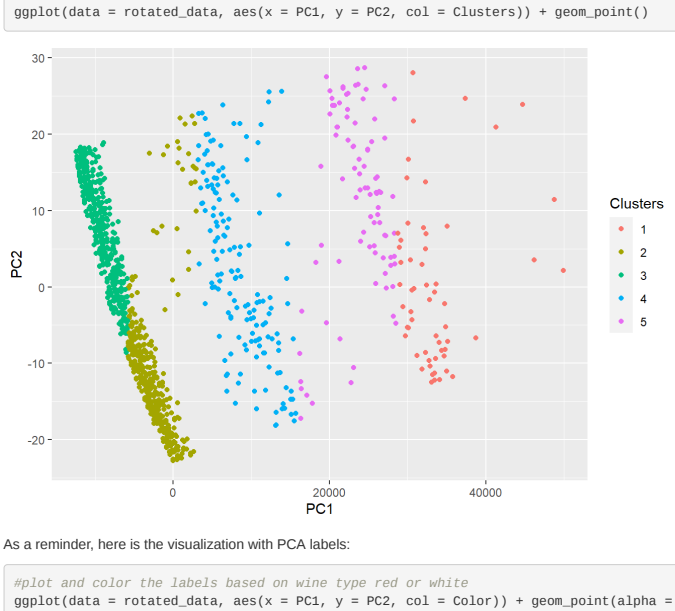
```
# Evaluation method
train_control = trainControl(method = "cv", number = 10)
# Fit the model
tree1 <- train(smoker ~., data = insurance, method = "rpart", trControl = train_control)
#evaluate the fit
tree1
```

```
## CART
##
## 1201 samples
##    6 predictor
##    2 classes: 'no', 'yes'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1082, 1082, 1081, 1081, 1080, 1080, ...
## Resampling results across tuning parameters:
##
##   cp           Accuracy   Kappa
##   0.008196721  0.9601024  0.8791186
##   0.092213115  0.9417336  0.8372919
##   0.651639344  0.8566281  0.3827625
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.008196721.
```

Accuracy seems to be comparable in decision trees and SVM, although SVM is consistently more accurate (96%+).

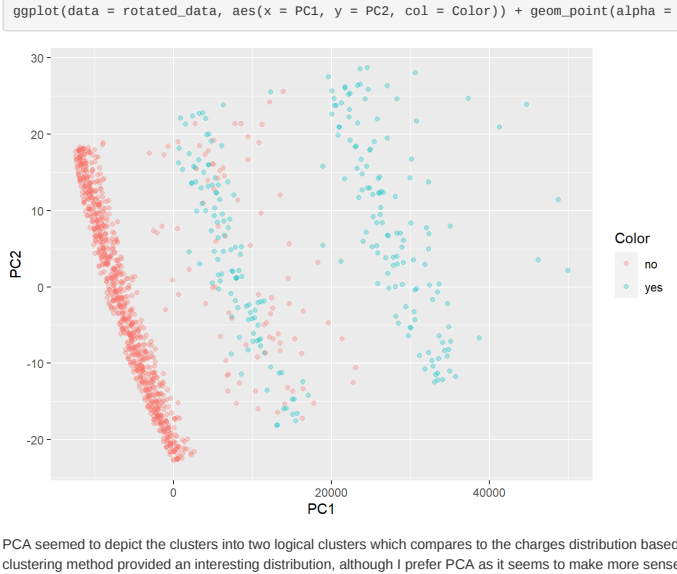
Now we can visualize the labels for KNN as we did previously with PCA:

```
# Assign clusters as a new column
rotated_data$Clusters = as.factor(fit$cluster)
# Plot and color by labels
ggplot(data = rotated_data, aes(x = PC1, y = PC2, col = Clusters)) + geom_point()
```



As a reminder, here is the visualization with PCA labels:

```
#plot and color the labels based on wine type red or white
ggplot(data = rotated_data, aes(x = PC1, y = PC2, col = Color)) + geom_point(alpha = 0.3)
```



PCA seemed to depict the clusters into two logical clusters which compares to the charges distribution based on your smoking habit. The KNN clustering method provided an interesting distribution, although I prefer PCA as it seems to make more sense compared to our exploratory work.

g. Evaluation

Using the better classifier from the previous step, perform a more sophisticated evaluation using the tools of Week 9. Specifically, (1) produce a 2x2 confusion matrix (if your dataset has more than two classes, bin the classes into two groups and rebuild the model), (2) calculate the precision and recall manually, and finally (3) produce an ROC plot (see Tutorial 9). Explain how these performance measures makes your classifier look compared to accuracy.

So I was supposed to use this step to predict whether the person is a smoker or not, but I had some issues trying to run the model with this dataset. Therefore, I decided to, for the sake of practice, determine age based on age groups. I will create a new column by binning ages into groups and creating a new column for them. While binning the classes into 2 groups is what the question is asking for, this didn't work with my dataset. Therefore, I've decided to create 5 bins instead based on age groups, which is more logical for the sake of this example.

First, let's create new bins for those age groups from 20 to 60+:

```
#create a new file for the modified dataset
myinsurance <- insurance

#remove regions
myinsurance <- myinsurance %>% select(-c(region))

#bin age groups into 5 different bins
myinsurance <- myinsurance %>% mutate(agegroup = cut(age,breaks=c(-Inf, 29, 39, 49, 59, Inf), labels=c("twenties"
, "thirties", "fourties", "fifties", "sixtiesplus")))

#now remove age
myinsurance <- myinsurance %>% select(-c(age))

head(myinsurance)
```

```
## # A tibble: 6 x 6
##   sex      bmi children smoker charges agegroup
##   <chr>  <dbl>    <dbl> <chr>    <dbl> <fct>   
## 1 male    33          3 no       4449. twenties
## 2 male    22.7        0 no      21984. thirties
## 3 male    28.9        0 no       3867. thirties
## 4 female  25.7        0 no       3757. thirties
## 5 female  33.4        1 no       8241. fourties
## 6 female  27.7        3 no       7282. thirties
```

Let's create a 70-30 train test split:

```
#set seed
set.seed(123)
# Partition the data
index = createDataPartition(y=myinsurance$agegroup, p=0.7, list=FALSE)
# Everything in the generated index list
train_set = myinsurance[index,]
# Everything except the generated indices
test_set = myinsurance[-index,]
```

Now let's fit the model:

```
# Fit the model using the training set
svm_split <- train(agegroup ~., data = train_set, method = "svmLinear")

# Predict with test set
pred_split <- predict(svm_split, test_set)

train_control_boot = trainControl(method = "boot", number = 100)

# Fit the model
svm <- train(agegroup ~., data = myinsurance, method = "svmLinear",
             trControl = train_control_boot)
# Evaluate fit
svm
```

```
## Support Vector Machines with Linear Kernel
##
## 1201 samples
##    5 predictor
##    5 classes: 'twenties', 'thirties', 'fourties', 'fifties', 'sixtiesplus'
##
## No pre-processing
## Resampling: Bootstrapped (100 reps)
## Summary of sample sizes: 1201, 1201, 1201, 1201, 1201, 1201, ...
## Resampling results:
##
##   Accuracy   Kappa
##   0.4354543  0.2728205
##
## Tuning parameter 'C' was held constant at a value of 1
```

Now we can generate a confusion matrix:

```
# Generate confusion matrix for the test set
cm <- confusionMatrix(test_set$agegroup, pred_split)
cm
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    twenties thirties fourties fifties sixtiesplus
##   twenties          60       15        5       4           0
##   thirties          40       24        7       6           0
##   fourties          19       29       21      12           2
##   fifties            2        4       22      45           8
##   sixtiesplus        1        1        5      23           4
##
## Overall Statistics
##
##                Accuracy : 0.429
##                  95% CI : (0.3772, 0.482)
##     No Information Rate : 0.3398
##     P-Value [Acc > NIR] : 0.0002759
##
##                   Kappa : 0.266
##
##  Mcnemar's Test P-Value : 7.573e-07
##
## Statistics by Class:
##
##                      Class: twenties Class: thirties Class: fourties
## Sensitivity                   0.4918         0.32877          0.3500
## Specificity                   0.8987         0.81469          0.7926
## Pos Pred Value                0.7143         0.31169          0.2530
## Neg Pred Value                0.7745         0.82624          0.8587
## Prevalence                    0.3398         0.20334          0.1671
## Detection Rate                0.1671         0.06685          0.0585
## Detection Prevalence          0.2340         0.21448          0.2312
## Balanced Accuracy             0.6953         0.57173          0.5713
##                      Class: fifties Class: sixtiesplus
## Sensitivity                  0.5000            0.28571
## Specificity                  0.8662            0.91304
## Pos Pred Value               0.5556            0.11765
## Neg Pred Value               0.8381            0.96923
## Prevalence                   0.2507            0.03900
## Detection Rate               0.1253            0.01114
## Detection Prevalence         0.2256            0.09471
## Balanced Accuracy            0.6831            0.59938
```

We can view the scoring metrics by class:

```
# Store the byClass object of confusion matrix as a dataframe
metrics <- as.data.frame(cm$byClass)
# View the object
metrics
```

```
##                    Sensitivity Specificity Pos Pred Value Neg Pred Value
## Class: twenties      0.4918033   0.8987342      0.7142857      0.7745455
## Class: thirties      0.3287671   0.8146853      0.3116883      0.8262411
## Class: fourties      0.3500000   0.7926421      0.2530120      0.8586957
## Class: fifties       0.5000000   0.8661710      0.5555556      0.8381295
## Class: sixtiesplus   0.2857143   0.9130435      0.1176471      0.9692308
##                    Precision    Recall        F1 Prevalence Detection Rate
## Class: twenties    0.7142857 0.4918033 0.5825243 0.33983287     0.16713092
## Class: thirties    0.3116883 0.3287671 0.3200000 0.20334262     0.06685237
## Class: fourties    0.2530120 0.3500000 0.2937063 0.16713092     0.05849582
## Class: fifties     0.5555556 0.5000000 0.5263158 0.25069638     0.12534819
## Class: sixtiesplus 0.1176471 0.2857143 0.1666667 0.03899721     0.01114206
##                    Detection Prevalence Balanced Accuracy
## Class: twenties              0.23398329         0.6952687
## Class: thirties              0.21448468         0.5717262
## Class: fourties              0.23119777         0.5713211
## Class: fifties               0.22562674         0.6830855
## Class: sixtiesplus           0.09470752         0.5993789
```

Precision:

```
# Get the precision value for each class
metrics %>% select(c(Precision))
```

```
##                    Precision
## Class: twenties    0.7142857
## Class: thirties    0.3116883
## Class: fourties    0.2530120
## Class: fifties     0.5555556
## Class: sixtiesplus 0.1176471
```

Recall:

```
# Get the recall value for each class
metrics %>% select(c(Recall))
```

```
##                       Recall
## Class: twenties    0.4918033
## Class: thirties    0.3287671
## Class: fourties    0.3500000
## Class: fifties     0.5000000
## Class: sixtiesplus 0.2857143
```

Specifity:

```
# Get the specificity value for each class
metrics %>% select(c(Specificity))
```

```
##                    Specificity
## Class: twenties      0.8987342
## Class: thirties      0.8146853
## Class: fourties      0.7926421
## Class: fifties       0.8661710
## Class: sixtiesplus   0.9130435
```

F1 Score:

```
# Get the F1 score value for each class
metrics %>% select(c(F1))
```

```
##                           F1
## Class: twenties    0.5825243
## Class: thirties    0.3200000
## Class: fourties    0.2937063
## Class: fifties     0.5263158
## Class: sixtiesplus 0.1666667
```

And Balanced Accuracy:

```
# Get the balanced accuracy value for each class
metrics %>% select(c(`Balanced Accuracy`))
```

```
##                    Balanced Accuracy
## Class: twenties            0.6952687
## Class: thirties            0.5717262
## Class: fourties            0.5713211
## Class: fifties             0.6830855
## Class: sixtiesplus         0.5993789
```

Now, since we want to produce ROC plot, I'll need to go back to the original class we're predicting, which is smokers.

```
insurance$smoker <- as.factor(insurance$smoker)
# Partition the data
index = createDataPartition(y=insurance$smoker, p=0.7, list=FALSE)
# Everything in the generated index list
train_ins = insurance[index,]
# Everything except the generated indices
test_ins = insurance[-index,]
```

Now building a KNN model with cross validation:

```
# Set control parameter
train_control = trainControl(method = "cv", number = 10)
# Fit the model
knn <- train(smoker ~., data = train_ins, method = "knn", trControl = train_control, tuneLength = 20)
# Evaluate fit
knn
```

```
## k-Nearest Neighbors
##
## 841 samples
##   6 predictor
##   2 classes: 'no', 'yes'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 757, 757, 757, 757, 757, 756, ...
## Resampling results across tuning parameters:
##
##   k   Accuracy   Kappa
##    5  0.9096359  0.7233259
##    7  0.9262745  0.7820089
##    9  0.9203221  0.7613353
##   11  0.9274650  0.7817188
##   13  0.9286555  0.7840713
##   15  0.9322269  0.7965628
##   17  0.9310364  0.7938353
##   19  0.9310364  0.7935091
##   21  0.9310224  0.7922355
##   23  0.9298459  0.7891862
##   25  0.9298599  0.7883158
##   27  0.9310364  0.7923514
##   29  0.9310364  0.7926576
##   31  0.9298459  0.7911929
##   33  0.9286555  0.7853271
##   35  0.9334034  0.7985509
##   37  0.9298319  0.7901421
##   39  0.9298319  0.7908169
##   41  0.9286415  0.7869785
##   43  0.9262605  0.7811315
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 35.
```

Now generate a confusion matrix:

```
# Evaluate the fit with a confusion matrix
pred_ins <- predict(knn, test_ins)
# Confusion Matrix
confusionMatrix(test_ins$smoker, pred_ins)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  no yes
##        no  272  15
##        yes   6  67
##
##                Accuracy : 0.9417
##                  95% CI : (0.9122, 0.9635)
##     No Information Rate : 0.7722
##     P-Value [Acc > NIR] : < 2e-16
##
##                   Kappa : 0.8275
##
##  Mcnemar's Test P-Value : 0.08086
##
##             Sensitivity : 0.9784
##             Specificity : 0.8171
##          Pos Pred Value : 0.9477
##          Neg Pred Value : 0.9178
##              Prevalence : 0.7722
##          Detection Rate : 0.7556
##    Detection Prevalence : 0.7972
##       Balanced Accuracy : 0.8977
##
##        'Positive' Class : no
##
```

Now we can generate an ROC table for our data:

```
library(pROC)
# Get class probabilities for KNN
pred_prob <- predict(knn, test_ins, type = "prob")
head(pred_prob)
```

```
##   no yes
## 1  1   0
## 2  1   0
## 3  0   1
## 4  1   0
## 5  1   0
## 6  1   0
```

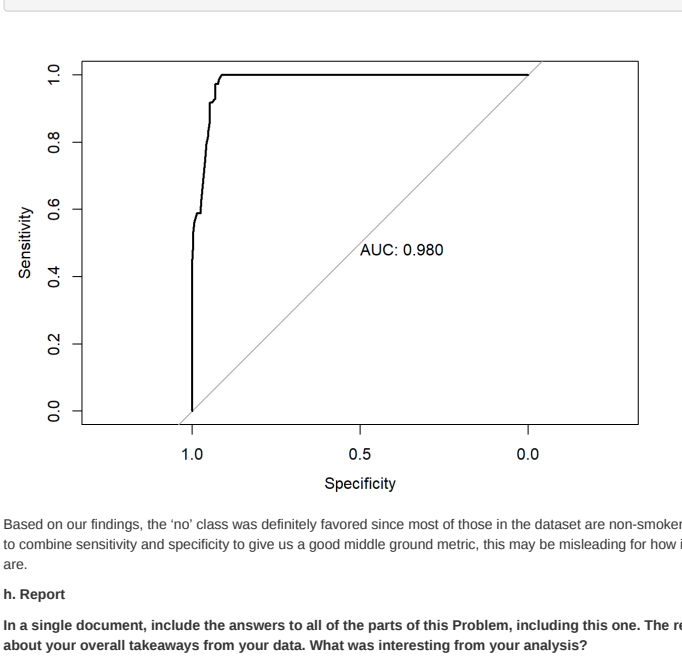
And now an ROC curve

```
# And now we can create an ROC curve for our model.
roc_obj <- roc((test_ins$smoker), pred_prob[,1])
```

```
## Setting levels: control = no, case = yes
```

```
## Setting direction: controls > cases
```

```
plot(roc_obj, print.auc=TRUE)
```



Based on our findings, the 'no' class was definitely favored since most of those in the dataset are non-smokers. While the AUC metric is supposed to combine sensitivity and specificity to give us a good middle ground metric, this may be misleading for how imbalanced the number of smokers are.

h. Report

In a single document, include the answers to all of the parts of this Problem, including this one. The report component specifically is about your overall takeaways from your data. What was interesting from your analysis?

So to summarize:

1. There was a clear imbalance in the number of smokers versus the number of nonsmokers in the dataset, which made for a challenging clustering task.
2. Smoking could be an indicator of insurance costs increasing. That is, there is a positive correlation between smoking and the increase of insurance charges, which was part of our hypothesis.
3. The nonsmokers were rare in each age group, and there was no true correlation between age and smoking habits.
4. Finally, our model struggled to predict age groups based on the data provided with about 50% accuracy reported, on average.

i. Reflection

The final section of the report is a (short) paragraph reflecting on the course as a whole and what you have learned. The goal is not actually feedback for the course but to get you to think back about what you have learned and how your perspective on data science has changed.

I think one way that this course changed my perspective on data science is by showing me how, regardless of the data, there is a correlation that could be drawn somewhere with some head scratching and analysis. I now always have a pending hypothesis to set a tone for each project as it can provide me with some guidance. If there is no clear correlation, it is important to perform exploratory tasks to view possible correlations that may influence or create a new hypothesis. Based on that, I can run my analysis and modeling, then reach conclusions to prove or disprove my pending hypothesis. Overall, this course taught me the 'scientific method' that we learned about in elementary from a data science perspective. :)